COMP3821/9801

Workshop Week 2

Greedy Algorithms

Announcements.

Copyright Disclaimer. *Reproducing, publishing, posting, distributing, translating this document is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.*

Guided Problem

The demonstrators will solve the following problems. For the rest of the workshop, work in small groups to solve the remaining workshop exercises. There are optional problems at the end of this document for further practice.

Guided Problem

Let $\mathcal{I} = \{I_1, \ldots, I_n\}$ be a set of n intervals on the real line; the ith interval can be described by a pair of real numbers a_i, b_i such that interval $I_i = [a_i, b_i]$. A *clique cover* is a partitioning of the intervals $\{C_1, \ldots, C_k\}$ such that, whenever two intervals are in the same set C_i, they intersect in at least one point.

Describe and analyse a polynomial-time algorithm that computes the smallest clique cover of $\mathcal{I}$.

Guided Problem

Your friends are starting a security company that needs to obtain licenses for n different pieces of cryptographic software. Due to regulations, they can only obtain these licenses at the rate of at most one per month.

Each license is currently selling for a price of \$100. However, they are all becoming more expensive according to exponential growth curves: in particular, the cost of license j increases by a factor of $r_j > 1$ each month, where r_j is a given parameter. This means that if license j is purchased t months from now, it will cost $100 \cdot r_j^t$. We will assume that all the price growth rates are distinct; that is, $r_i \neq r_j$ for licenses $i \neq j$ (even though they start at the same price of \$100).

Given that the company can only buy at most one license a month, describe and analyse a polynomial-time algorithm that takes the n rates of price growth $r_1, \ldots, r_n$ and computes an order in which to buy the licenses so that the total amount of money spent is minimised.

Workshop Exercises

1. Let $\mathcal{I}$ be a set of n intervals on the real line. We say that a subset $\mathcal{I}' \subseteq \mathcal{I}$ of intervals is a *covering* of $\mathcal{I}$ if, for every pair of intervals $i, j \in \mathcal{I}$ that intersect, at least one of i and j is in $\mathcal{I}'$. Describe and analyse a polynomial-time algorithm that computes the smallest covering of $\mathcal{I}$.
2. Suppose we are to schedule print jobs on a printer. Each job j has associated a weight $w_j > 0$, representing how important the job is, and a processing time t_j, representing how long the job takes. A schedule σ is an ordering of the jobs that tell the printer how to process the jobs. Let C_j^σ be the completion time of job j under the schedule σ. Describe and analyse a polynomial-time algorithm that computes a schedule σ minimising the sum of weighted completion times; that is, we want to find a schedule σ that minimises $\sum_j w_j C_j^\sigma$.
3. You are running a small manufacturing shop with plenty of workers, but just a single milling machine. You have to produce n items; item i requires m_i machining time first, then p_i polishing time by hand; finally it is packaged by hand and

this takes c_i amount of time. The machine can mill only one item at a time, but your workers can polish and package in parallel as many objects as you wish. Describe and analyse a polynomial-time algorithm that determines the order in which the items should be machined so that the whole production is finished as quickly as possible.

4. A battle takes place between n heroes and m monsters. Let h_j denote the starting amount of health points of monster j, where $1 \leq j \leq m$. During one round of the battle, each hero attacks a monster of their own choice, causing one damage to the monster's health points. If a monster's number of health points reduces to zero at any time, they are killed. At the end of each round, all monsters still alive will gain k health points; therefore, a monster can exceed h_j health points. The monsters will not attack any of the heroes.

   The heroes win the battle if they manage to kill all of the monsters.

   (a) Suppose that $k < n$. Describe a strategy that chooses which monster(s) to attack in each round, in order to guarantee victory for the heroes in the fewest number of rounds as possible.

   (b) Prove that your algorithm guarantees that the heroes win and prove that the strategy in part (a) guarantees that the heroes win in the fewest number of rounds possible.

   (c) Now, suppose that $k \geq n$. Under what circumstance(s) can the heroes win the battle? Briefly explain why.

Optional Exercises

These exercises will not contain solutions. We encourage that you post on the forums to discuss the approaches for these problems. Additionally, you may find the hints helpful.

5. Let $\mathcal{I}$ be a set of n intervals on the real line. We say that a subset $\mathcal{I}' \subseteq \mathcal{I}$ of intervals is *dominating* of $\mathcal{I}$ if every interval in $\mathcal{I}$ intersects with at least one interval in $\mathcal{I}'$. Describe and analyse a polynomial-time algorithm that computes the smallest dominating set of $\mathcal{I}$.

6. Let A and B be two arrays, each containing n positive integers. We define the *dot product* of A and B as the sum of the product of the ith element of A and B; in other words, we have that

$$A \cdot B = \sum_{i=1}^{n} A[i] \cdot B[i].$$

   Given arrays A and B, you want to compute the maximum dot product that can be achieved by (possibly) permuting A and B. In other words, you are allowed to permute the elements of A and B to obtain two new arrays A' and B', where the dot product of the permutations is $A' \cdot B' = \sum_{i=1}^{n} A'[i] \cdot B'[i]$. Describe and analyse a polynomial-time algorithm that computes the maximum dot product achieved by any permutation.

7. You are given a set of n jobs where each job i has deadline $D[i] \geq 1$ and profit $P[i] > 0$. Only one job can be scheduled at a time, and each job takes one unit of time to complete. We can only earn profit for a job if and only if the job is completed by its deadline. Describe and analyse a polynomial-time algorithm to return a subset of jobs that maximises the profit.

8. You are given a collection of n white dots and n black dots on the real number line and your task is to pair each white dot with some black dot in a way such that:

   - each white dot is paired with *exactly* one black dot and every black dot is paired with *exactly* one white dot, and
   - the overall sum of the distance of the pair of white and black dots is minimised.

   The input to the problem is a set of locations $W[1..n]$ of white dots and a set of locations $B[1..n]$ of black dots. Describe and analyse a polynomial-time algorithm to return a set of n pairs $\{(W'[1], B'[1]), \ldots, (W'[n], B'[n])\}$ such that

$$\sum_{i=1}^{n} |W'[i] - B'[i]|$$

   is minimised.

9. Your friend is working as a camp counsellor, and he is in charge of organising activities for a set of high school age campers. One of his plans is the following mini-triathlon exercise: each contestant swims 20 laps of a pool, then cycles 10km, and then runs 3km. The plan is to send the contestants out in a staggered fashion, via the following rule:

   the contestants must use the pool one at a time.

In other words, the first contestant swims the 20 laps, gets out, and starts biking. As soon as the first contestant is out of the pool, the second contestant begins swimming the 20 laps; as soon as (s)he is out and starts cycling, a third contestant begins swimming, and so on.

Each contestant has a projected *swimming time* (the expected time it will take them to complete the 20 laps), a projected *cycling time* (the expected time it will take them to complete 10km of cycling), and a projected *running time* (the expected time it will take them to complete the 3km of running).

Your friend wants to decide on a *schedule* for the triathlon: an order in which to sequence the starts of the contestants. Let the *completion time* of a schedule denote the earliest time at which all contestants finish all three legs of the triathlon, assuming they each spend exactly their projected swimming, biking, and running times on the three parts.

Describe and analyse a polynomial-time algorithm to find the best order to send these contestants out, minimising the overall completion time.

10. Consider the following problem.

    Let $[n] := \{1, \ldots, n\}$. What is the size of the largest subset $S \subseteq [n]$ such that every pair of elements in S is coprime?

    Let P_n denote the set of primes in $[n]$. Let $S = \{1\} \cup P_n$. Prove that S produces a set of coprime integers of maximum cardinality.

    Note. *This implies that the natural greedy algorithm of "include x iff. x is prime or x is 1" returns a subset of maximum cardinality.*

11. The VertexCover problem states: given an undirected unweighted graph $G = (V, E)$, what is the size of the *smallest* subset $V' \subseteq V$ of vertices such that each edge in E is incident to at least one vertex in V'?

    On general graphs, this problem is NP-hard. However, VertexCover is solvable in polynomial-time on acyclic graphs (i.e. forests) with the following algorithm.

    - Initialise our vertex cover V' to be empty.
    - For each leaf vertex v in G, add its neighbour u to V', and delete u, v and all incident edges from G.
    - Once the edge set of G is empty, return V'.

    You are asked to prove that the subset of vertices produced by the algorithm forms an optimal vertex cover.

    (a) Prove that V' is a valid vertex cover.

    (b) Prove that V' is an optimal vertex cover and hence, conclude that the algorithm is correct.

12. Alice wants to throw a party and is deciding who to invite. She has n people to choose from, and she has created a list consisting of pairs of people who know each other. She wants to invite as many people as possible, subject to the following two constraints.

    - Each person should have at least five other people whom they know.

- Each person should also have at least five other people whom they do not know.

Formally, you are given the list of n people and the list of all pairs of people who *mutually* know each other; that is, you may assume that if X knows Y, Y also knows X. To make things simpler, we can model the problem as a graph. Each person is a vertex, and an *undirected* edge connects two people X, Y if and only if X and Y mutually know each other. Therefore, the input can be represented by a graph $G = (V, E)$ and an adjacency matrix $A[1..n, 1..n]$.

(a) Describe and analyse a polynomial-time algorithm that returns a subset of people with maximum cardinality that satisfies the two constraints.

(b) Prove that the optimal subset is unique.

(c) Hence or otherwise, prove that the algorithm you proposed in part (a) is correct.

13. Let C be a set of n circles on the two-dimensional plane, where the ith circle is defined by a point (x_i, y_i) and a radius r_i. Furthermore, suppose that one can shoot a ray from the origin that does not intersect any circles. Describe and analyse an $O(n \log n)$ algorithm that computes the minimum number of rays from the origin that intersect every circle.

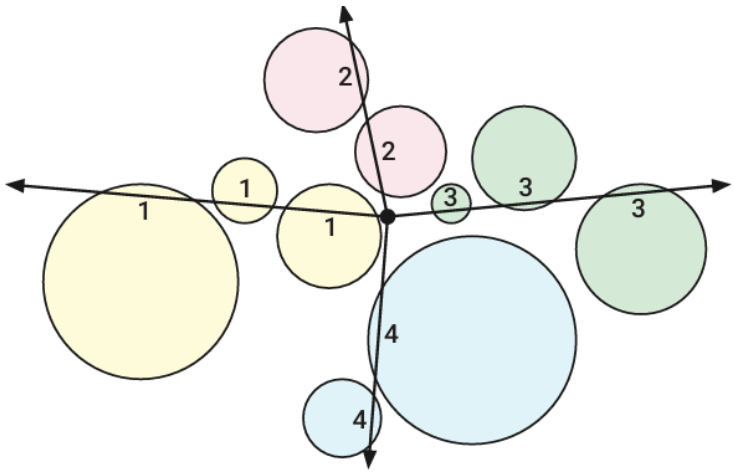

Figure 1: *A total of nine balloons popped by four rays.*

14. Let A be a $2 \times n$ array of positive real numbers such that the elements in each column sum to 1. Describe and analyse an $O(n \log n)$ algorithm to pick one number in each column such that the sum of the numbers chosen in each row does not exceed $\frac{n+1}{4}$.

For example, consider the following 2×8 array.

0.4	0.7	0.9	0.2	0.6	0.4	0.3	0.1
0.6	0.3	0.1	0.8	0.4	0.6	0.7	0.9

We can choose the following configuration (the configuration is highlighted in red).

0.4	0.7	0.9	**0.2**	**0.6**	**0.4**	**0.3**	**0.1**
0.6	**0.3**	**0.1**	0.8	0.4	0.6	0.7	0.9

The chosen numbers on the first row give us

$$0.4 + 0.2 + 0.6 + 0.4 + 0.3 + 0.1 = 2 < 2.25 = \frac{9}{4},$$

while the chosen numbers on the second row give us

$$0.3 + 0.1 = 0.4 < 2.25 = \frac{9}{4}.$$

Hints

5. Try to reduce this to a problem you have seen before.
6. Since all of the integers are positive, a large integer should never pair with a small integer; as a follow up, can you solve this problem if the integers *weren't* positive?
7. You want to take the jobs that give you the most profit; how should you schedule this job?
8. It might not always be optimal to pair the *closest* set of points.
9. See Problem 3.
10. There are at least two ways to show optimality; one uses a standard exchange argument and one uses an injective argument. Try and define an injection.
11. For the optimality proof, start with an optimal solution; if it does not contain the neighbour of a leaf vertex, what must it contain?
12. To prove uniqueness, suppose you have sets X and Y of maximal cardinality. Show that $X \cup Y$ is feasible. Then it suffices to show that the output produced by the greedy solution is *maximal*.
13. How does the fact that we can shoot a ray from the origin help us here?
14. You might need a bit of calculus to show that the greedy algorithm is correct.